

AI 开发说明书

SmartCinema 智能影院选座系统

项目：SmartCinema 智能影院选座系统 组员：张颂（成员 A）、周皓洋（成员 B）、黄和（成员 C）、熊天齐（成员 D）

本文档记录开发过程中 AI 协同的情况。四名组员在产品决策、规则设计、架构选型和体验调优上承担了主要工作，AI 仅在样板代码、API 查询、细节实现等技术层面提供辅助。下面按模块说明设计思路与各自的工作。

1 对 AI 协作的定位

组内开发前达成共识：产品层面的判断必须由人完成。具体到本项目，四名组员给自己定了三条原则：

1. 推荐规则、评分权重、热度模型等影响用户体验的核心设计，先由负责人想清楚方案，再交付实现。
2. AI 提供的代码片段必须经过理解、修改和测试，不直接整段采用。
3. 每个模块都说明清楚”人做了什么决策、AI 辅助了哪些技术细节”。

2 张颂（成员 A）——智能推荐引擎与 AI 顾问

2.1 推荐规则：硬约束而非软扣分

作业明确要求”少年不能坐前三排、老人不能坐最后三排”。张颂在方案选型时否决了软扣分（即违规座位仅扣分、仍可能被推荐）的做法，理由是”不能”属于硬约束，扣分不足时仍会把少年安排到前三排，既违反作业要求，也不符合真实场景（家长不会让小孩坐第一排仰头观影）。

最终方案是两阶段：先用 `violatesRule()` 与 `sameRowRuns()` 把违反规则的座位直接排除，再在剩余座位中打分排序。这种”先过滤再评分”的结构由张颂确定，相关函数的实现细节由 AI 辅助编写。

团体票部分同样体现了人的判断。作业原文是”成员不能分开，要坐同一排”。为提高推荐成功率，曾考虑过前后排拼合的兜底逻辑——同排不够时拆到相邻两排。张颂删

掉了这一兜底，因为“同一排”是硬性要求，给出不符合要求的散座方案比提示“换更大的厅”对用户体验损害更大：用户信任推荐、到场却发现没坐在一起，体验会彻底崩塌。

2.2 票型加成的参数取舍

各票型的加成权重是张颂根据座位图模拟后确定的：

- 情侣票：靠近中轴线加分，系数取 25，因为情侣观影最在意并排居中。
- 家庭票：中后排固定加 18 分，带孩子家庭更需要稳定视角而非极致居中。
- 个人票：不额外加成，直接按体验分排序，对应作业“成年人可以随意坐”。

这些数字是张颂在纸上画座位图、代入几种典型情况推演后定下的；AI 负责协助完成均值计算与归一化的代码实现。

2.3 AI 顾问：本地规则解析而非调用大模型

AI 顾问（加分项）的目标是让用户用一句话描述需求即可选座，例如“两个大学生周末看科幻片”。关于解析策略，张颂决定不调用在线大模型 API：一是作业要求纯前端、可直接打开 `index.html` 运行，外部依赖会破坏这一约束；二是真实购票场景下用户输入的语义模式有限（票型、人数、年龄、片种），关键词匹配完全够用。

关键词词典与匹配规则由张颂设计，AI 辅助编写正则与字符串处理。在测试阶段，张颂发现并修正了两类解析缺陷：

1. “两个人”误判为个人票：原匹配规则中“一个人”作为个人票关键词，会误匹配“两个人”内部子串。修正方式是将个人票关键词改为“我自己 | 独自 | 单独一人”，并加入边界处理。
2. 中文数字未识别：“八个同事”“一家四口”中的“八”“四”未被转成人数。张颂补充了 `cnToNum()` 函数处理中文数字，并增加“一家 X”的特殊匹配模式。

上述修正以 10 组覆盖典型表达的测试用例验证通过。这是人在语义边界处理上补全技术实现细节的典型环节。

2.4 推荐理由的可读性

推荐理由要求逐条说明命中规则——有几位少年因此避开前三排、有几位老人因此避开后三排、团体锁定在第几排连续。文案模板由张颂拟定，AI 协助拼接字符串，最终措辞经张颂调整，去掉了“为您”“已锁定”等客服化表达。

3 周皓洋（成员 B）——热度地图与视觉设计

3.1 热度模型：非对称的双高斯分布

热度地图的核心是热度计算公式，由周皓洋重新设计。早期方案仅依据座位到影厅中心的距离，对称分布。周皓洋认为这与真实选座行为不符：观众对左右居中更敏感，对前后排相对宽容（中间几排差异不大，但前排与最后排明显偏冷）。

因此改为行列两个独立高斯：行方向黄金排在中间偏后（**总排数** * 0.55），列方向用更窄的高斯让左右居中权重更集中；两者加权求和时列权重（0.55）高于行（0.45），对应“更在意左右居中”这一判断。

此外周皓洋补充了若干真实场景因素：周末整体热度上浮、走道两侧因进出便利略加分、已售座位本身热度更高等。这些因素的选取来自实际观影经验，技术实现由 AI 协助完成。

3.2 颜色映射：HSL 插值替代 RGB 硬算

早期颜色映射采用 RGB 三通道分段计算，渐变存在断层、视觉发脏。周皓洋将其改为 HSL 色相插值，从蓝（240°）经黄（55°）到红（0°）单调过渡，并刻意绕开紫色色相段，使渐变平滑。该改动源于调色阶段的视觉判断。

3.3 自设计 IMAX 弧形厅

作为加分项，周皓洋新增了 IMAX 弧形巨幕厅。原座位几何函数中弧度为写死常量，周皓洋将其改为读取 `hall.curve` 参数（标准厅 12、IMAX 厅 30），使各类影厅共用同一套几何逻辑。这一重构的考量是避免分叉实现：若为 IMAX 单独写一段几何代码，后续新增影厅类型需改动多处。参数化之后仅需在配置中增加一项即可扩展。

3.4 视觉风格的整体把控

整体视觉方向（深蓝黑背景、电光青强调）由周皓洋确定，参考了 Apple 与 Tesla 的暗色界面。在选择座位色系时，周皓洋否决了以大红大黄提高辨识度的方案，认为这与“影院暗场氛围”冲突；最终以深色背景模拟影院熄灯环境、让座位色块高对比跳出，视觉重心落在座位本身。

色盲友好模式将红绿替换为蓝紫，该决策基于避免红绿色觉混淆的色彩理论考量，具体色值由 AI 协助配置。

4 黄和（成员 C）——手动选座与无障碍

4.1 命中检测：先做对再做快

Canvas 命中检测（判断点击落在哪个座位）是手动选座的基础。黄和在实现节奏上选择先以 $O(n)$ 遍历方案保证命中准确，待功能稳定后再优化为“先按纵坐标定位排、再按横坐标定位列”的 $O(1)$ 方案。这种“先对再快”的节奏由黄和把握，目的是避免过早优化引入 bug。

拖拽框选（加分项）中“哪些座位被框选”的判定逻辑由黄和设计——判断座位中心是否落在选择矩形内；边界归一化（处理拖拽方向）等细节由 AI 协助实现。

4.2 坐标转换的推导

Canvas 的内部分辨率属性与 CSS 显示尺寸是两套体系，叠加 Retina 屏 DPR、画布平移与缩放后，坐标转换容易出错。黄和手动推导了完整的逆变换（逻辑坐标 = (鼠标坐标 - 偏移) / 缩放），并在草稿上以坐标系验证。AI 提供的相关函数因未考虑项目自定义的偏移与缩放，未被直接采用。

4.3 无障碍：不止于换颜色

无障碍模式包括大字体、高对比度、色盲友好、语音提示四项。AI 对无障碍的理解偏向“切换 CSS 类名改变颜色”，黄和认为不够：

- 大字体联动画布：CSS 字号改变不影响 Canvas 内由 JS 绘制的文字。黄和在 `drawSeats` 中读取当前模式，相应放大 Canvas 字号与座位半径。该联动在测试中发现“Canvas 内文字仍偏小”后补充。
- 高对比度加粗边框：仅改颜色对低视力用户不足，座位边框需更明显。
- 语音与偏好持久化：语音提示采用 Web Speech API，对每次操作播报；无障碍偏好写入 LocalStorage，避免老人每次进入重复开启。

5 熊天齐（成员 D）——架构、评分、订单与实时同步

5.1 架构先行：事件总线与单一状态源

项目有六个模块都会修改座位状态（推荐、手动选座、订单、热度等）。熊天齐在开工阶段首先确立的是 EventBus（事件总线）加 SeatData（单一状态源）的架构，而非急于编写功能。

所有座位状态集中于 `SeatData` 管理，任何模块修改均通过事件 `seats:changed` 通知，监听该事件的 UI 自动刷新，模块间不直接耦合。该架构决策由熊天齐做出，考量是四人并行开发时若不分模块、不分状态管理，合并代码将冲突频繁。EventBus 的发布-订阅实现由 AI 协助编写。

5.2 评分权重的直觉校验

评分涉及视角、距离、周围空位三个维度。熊天齐将权重定为视角 0.45、距离 0.35、空位 0.20——视角权重最高，因为视角偏差对观影体验影响最大（坐最边上需持续扭头），空位影响相对较小。

这些权重经 Excel 模拟验证：代入若干典型座位（黄金位、最边角、最后一排、被包围座位）的维度分数，套用不同权重比较总分排序是否符合直觉，调至“黄金位最高、边角最低、被包围次低”后定稿。等级阈值（80 极佳、60 优秀、其余一般）同样以保证三档区分度为原则确定。

5.3 订单状态机

订单状态包含已预订、已购票、已退票、已取消四态，对应作业要求的预订、取消预订、购票、退票四种操作。状态转移规则（各状态下可执行的操作）由熊天齐在纸上画出确认无死循环与非法跳转，LocalStorage 的 CRUD 封装由 AI 协助实现。

座位状态与订单状态的联动是难点。熊天齐采用“退票/购票后重建座位数据”(`makeSeats()` 重新读取 LocalStorage) 的方式，而非逐个修改座位状态，以保证一致性、避免遗漏。

5.4 实时同步：纯前端约束下的多人模拟

WebSocket 加分项面临的核心矛盾是：项目纯前端、无后端，真正的 WebSocket 需要服务器。熊天齐没有选择搭建 Node 服务，也没有放弃该加分项，而是采用浏览器原生的 BroadcastChannel API，在同源多标签页之间实现实时通信。开启两个标签页、各登录不同账号，即可模拟“多人同时选座”。

这一决策基于作业要求中“多人在线模拟”的措辞——BroadcastChannel 在纯前端约束下达成了实时、双向、事件驱动的多人效果。熊天齐在代码注释中说明：未来若接入真正的 WebSocket 服务器，仅需替换收发层，上层的 lock/sold/presence 逻辑无需改动，架构上预留了升级空间。

同步逻辑中，熊天齐额外处理了多人冲突：其他标签页正在选的座位，本地用户无法选中，避免两人抢同一座位。相关判定与命中检测的衔接由熊天齐完成。

6 小结

整体来看，AI 在本项目中主要承担三类技术辅助：编写样板代码（事件总线、增删改查、正则匹配）、查询 API 用法（BroadcastChannel、SpeechSynthesis 等）、协助实现既定方案的细节。

影响用户体验的关键决策——推荐规则用硬约束还是软罚分、热度如何分布、评分权重如何分配、实时同步采用何种技术方案、无障碍是否联动 Canvas——均由四名组员讨论后自行确定。方案选型、参数调优、边界处理、体验打磨等环节，人的判断主导了最终结果。

7 参考资料

- BroadcastChannel API ——MDN Web Docs
- Canvas API (getContext、createLinearGradient、devicePixelRatio) ——MDN Web Docs
- Web Speech API (SpeechSynthesisUtterance) ——MDN Web Docs
- HSL 色彩模型与渐变插值——色彩理论基础
- 《Don't Make Me Think》——Steve Krug（项目设计原则来源）
- 开发过程中使用 ChatGPT、Claude 辅助编写样板代码与排查技术问题